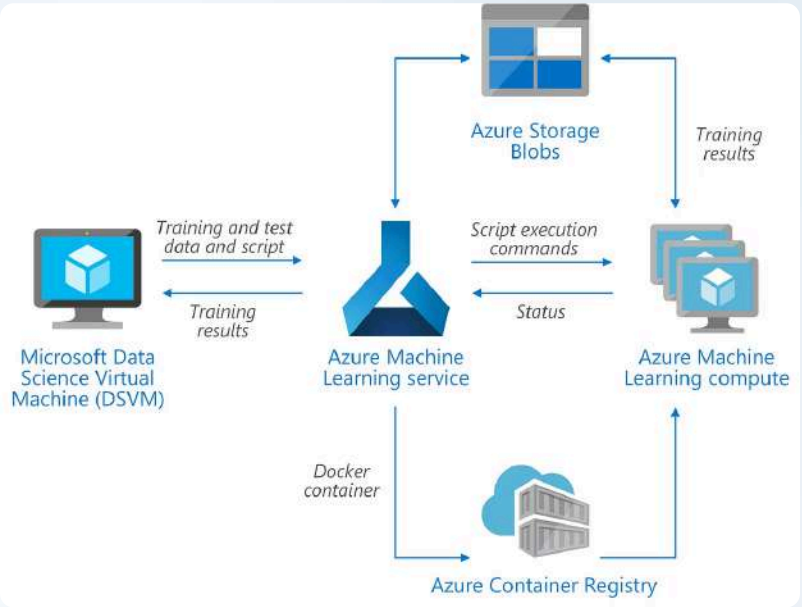
Azure Data Scientist Training — Day 3

Machine Learning Models, Evaluation Metrics, Experiment Tracking & Hands-On Labs

DP-100 CERTIFICATION PREP

DAY 3 OF TRAINING



Day 3 Session Overview

01

Theory Block

ML fundamentals, algorithms, evaluation metrics, and Azure ML experiment tracking concepts

02

Hands-On Labs

Lab 7: Model Training · Lab 8: Experiment Tracking · Lab 9: Hyperparameter Tuning

03

Wrap-Up & Takeaways

Review key concepts, cost optimization tips, and prepare for Day 4

What You'll Be Able to Do After Day 3

Train Models

Build and run ML models using Azure ML SDK and scikit-learn

Evaluate Performance

Apply classification and regression metrics to assess model quality

Track Experiments

Log metrics, compare runs, and manage model versions in Azure ML

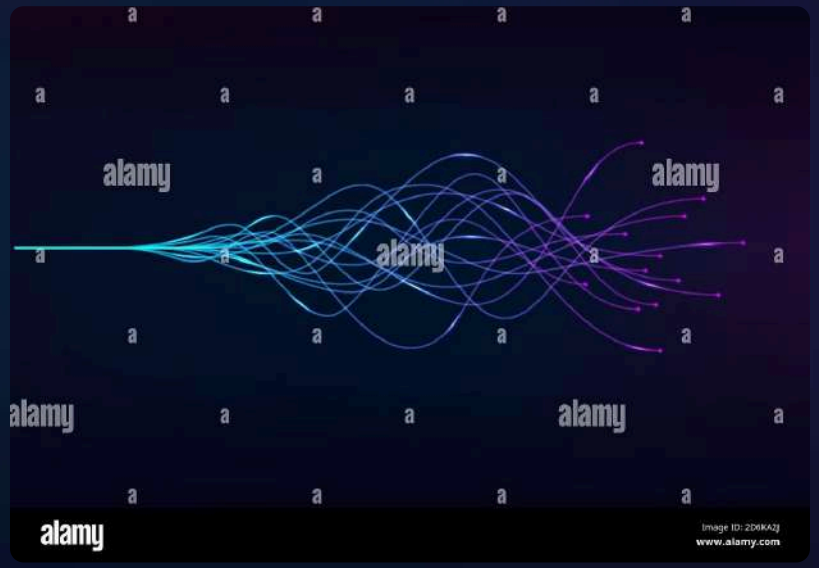
Tune Hyperparameters

Use HyperDrive to automatically find the best model configuration

CHAPTER 1

Machine Learning Fundamentals

Understanding how models learn from data — and why it matters for your business



What Is Machine Learning?

Machine learning enables systems to **learn patterns from data** and make decisions without being explicitly programmed for every scenario.

Instead of writing rules manually, you train a model on historical data — and it learns the rules itself.

Key insight: ML is most valuable when rules are too complex to hard-code, or when patterns change over time.

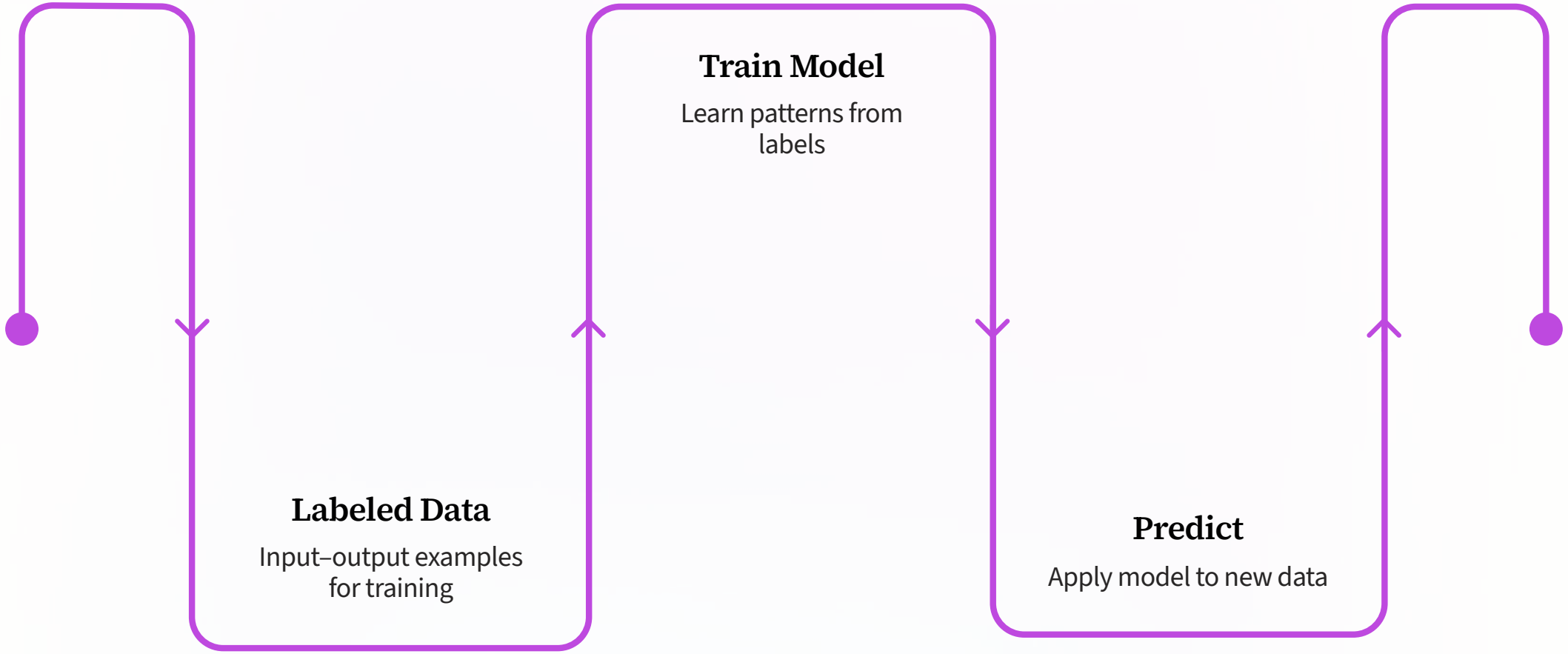
Traditional Programming

Data + Rules → Output

Machine Learning

Data + Output → Rules (Model)

Supervised Learning: The Foundation



In supervised learning, the model learns from **labeled examples** — pairs of inputs and known outputs. Once trained, it generalizes to make predictions on new, unseen data. This is the most common ML paradigm used in business applications.

Regression vs. Classification

How to Choose?

- Output is a **number** → Use Regression
- Output is a **category or label** → Use Classification
- Azure ML supports both natively across all compute types

ML in Real Business Applications

Churn Prediction

Telecom companies predict which customers will cancel — enabling proactive retention campaigns

Fraud Detection

Banks flag suspicious transactions in real time using classification models trained on historical fraud patterns

Recommendations

E-commerce platforms drive revenue by recommending products based on past behavior and purchase history

Demand Forecasting

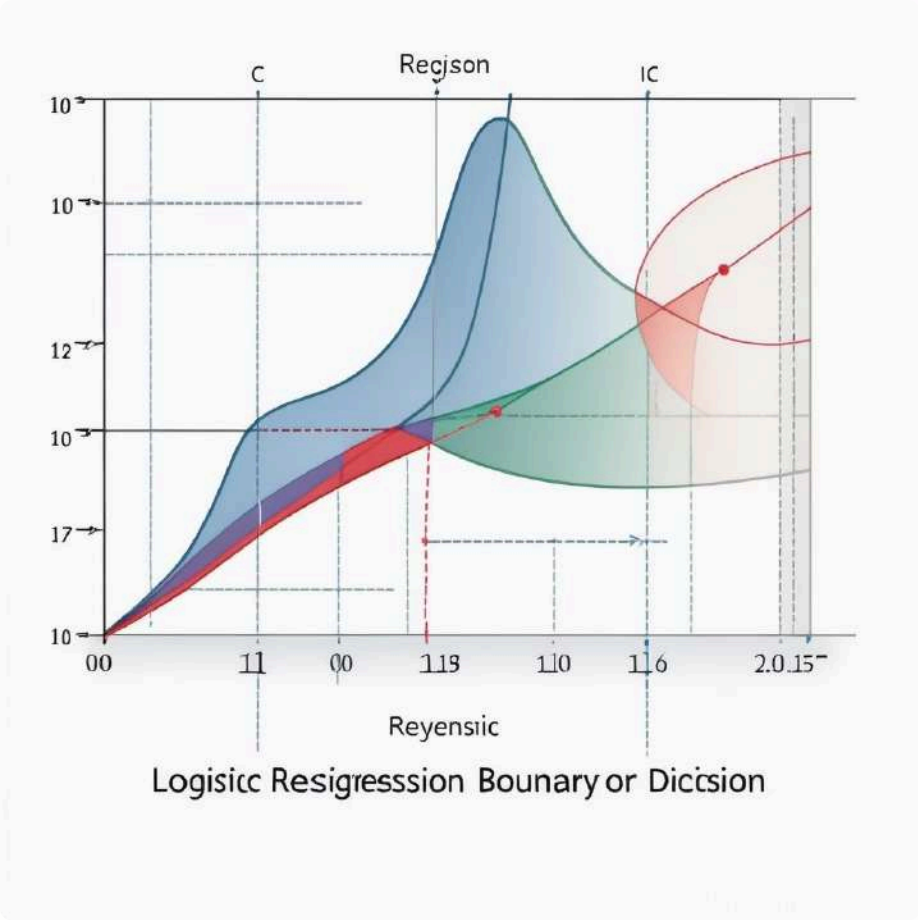
Retailers use regression models to predict inventory needs and optimize supply chain operations

CHAPTER 2

ML Algorithms in Azure ML

Understanding when to use which algorithm — and why it matters

Algorithm 1: Logistic Regression



What It Does

Despite the name, Logistic Regression is a **classification algorithm**. It predicts the probability that an input belongs to a specific class.

When to Use It

- Binary classification problems
- When you need **interpretable results**
- Good baseline model before trying complex algorithms

Example: Email spam detection — Is this email spam? (Yes/No)

Algorithm 2: Decision Trees

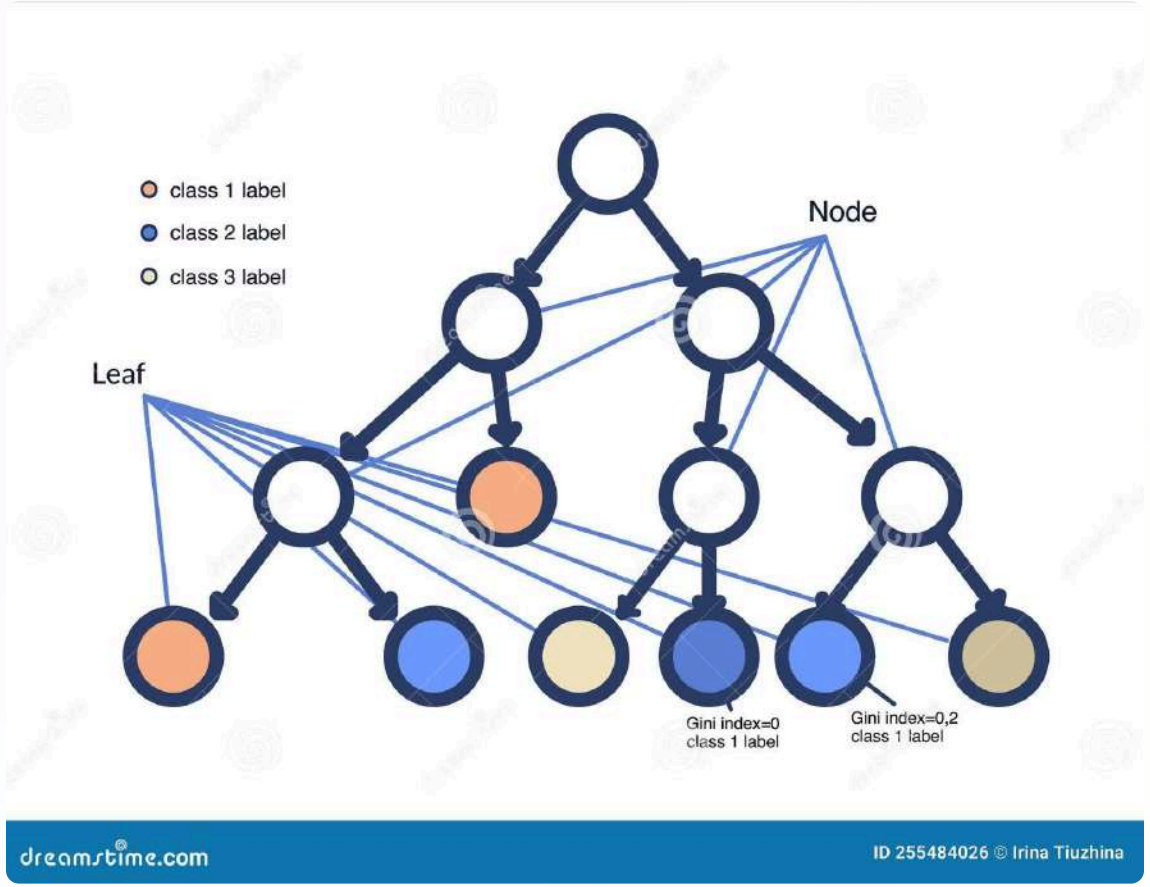
What It Does

A Decision Tree splits data using a series of **if-then rules**, forming a tree structure that's easy to visualize and explain.

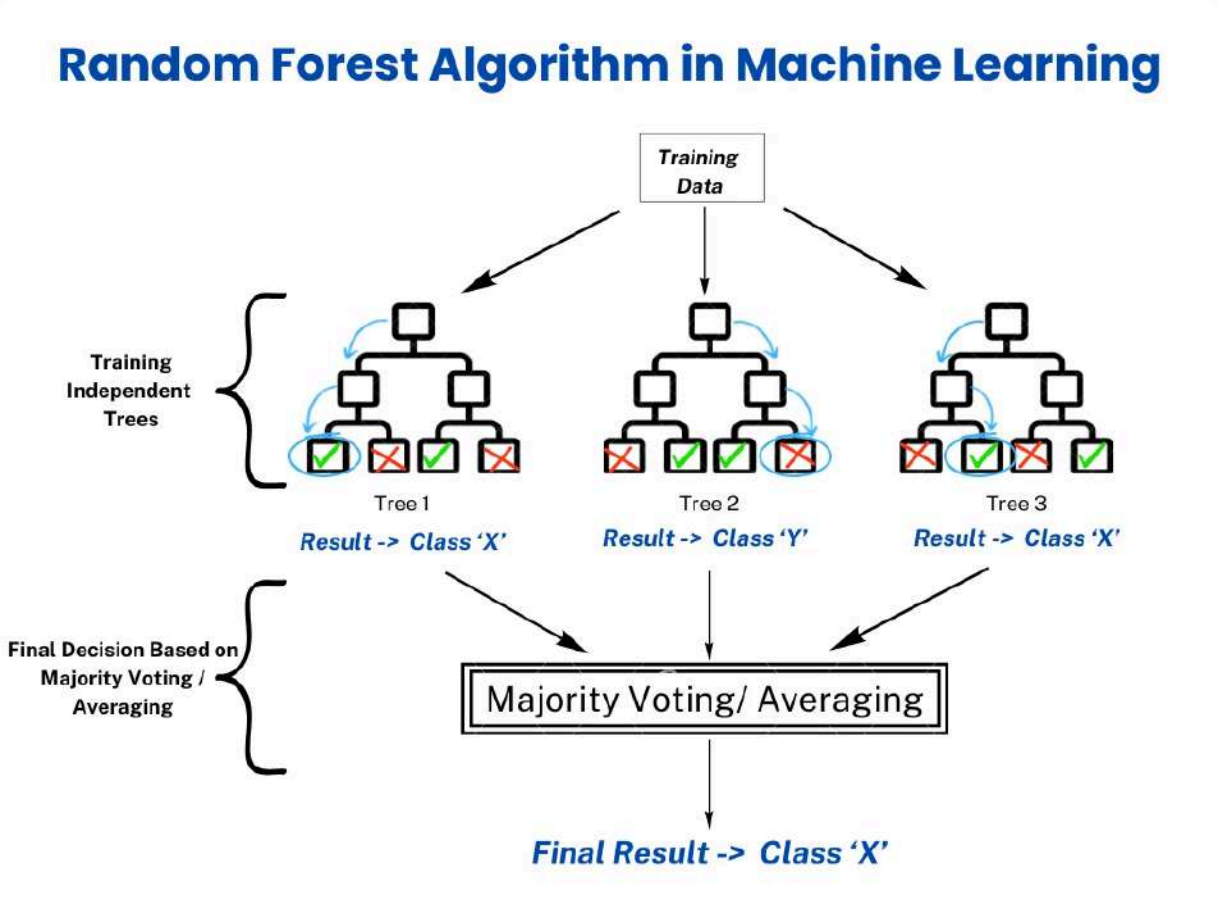
When to Use It

- When **explainability** is critical (e.g., regulated industries)
- Classification and regression tasks
- Small to medium datasets

Example: Credit approval — Income > $50K AND no defaults → Approve loan



Algorithm 3: Random Forest



What It Does

Random Forest builds **many decision trees** on random subsets of data, then aggregates their predictions — reducing overfitting and improving accuracy.

When to Use It

- High-accuracy classification or regression
- When individual trees overfit
- Feature importance analysis

Example: Customer churn — combines hundreds of trees for robust predictions

Algorithm 4: Gradient Boosting

What It Does

Gradient Boosting builds trees **sequentially** — each new tree corrects the errors of the previous one. It often delivers state-of-the-art performance.

When to Use It

- Structured/tabular data competitions
- When you need maximum predictive accuracy
- Classification and regression

Common implementations: XGBoost, LightGBM, Azure ML AutoML default

Algorithm Selection at a Glance

Logistic Regression

Fast, interpretable

Binary classification baseline

Decision Tree

Explainable rules

Great for regulated industries

Random Forest

High accuracy, robust

Handles noisy data well

Gradient Boosting

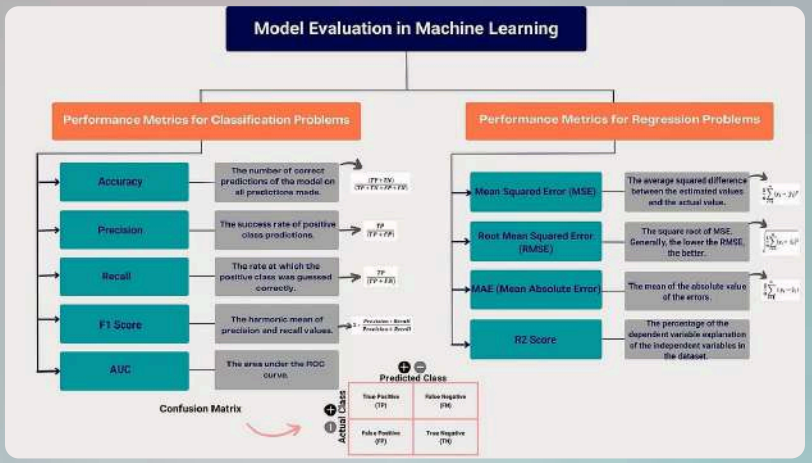
Best-in-class accuracy

Production ML workloads

CHAPTER 3

Model Evaluation Metrics

How do you know if your model is actually good? The answer depends on what you're predicting.



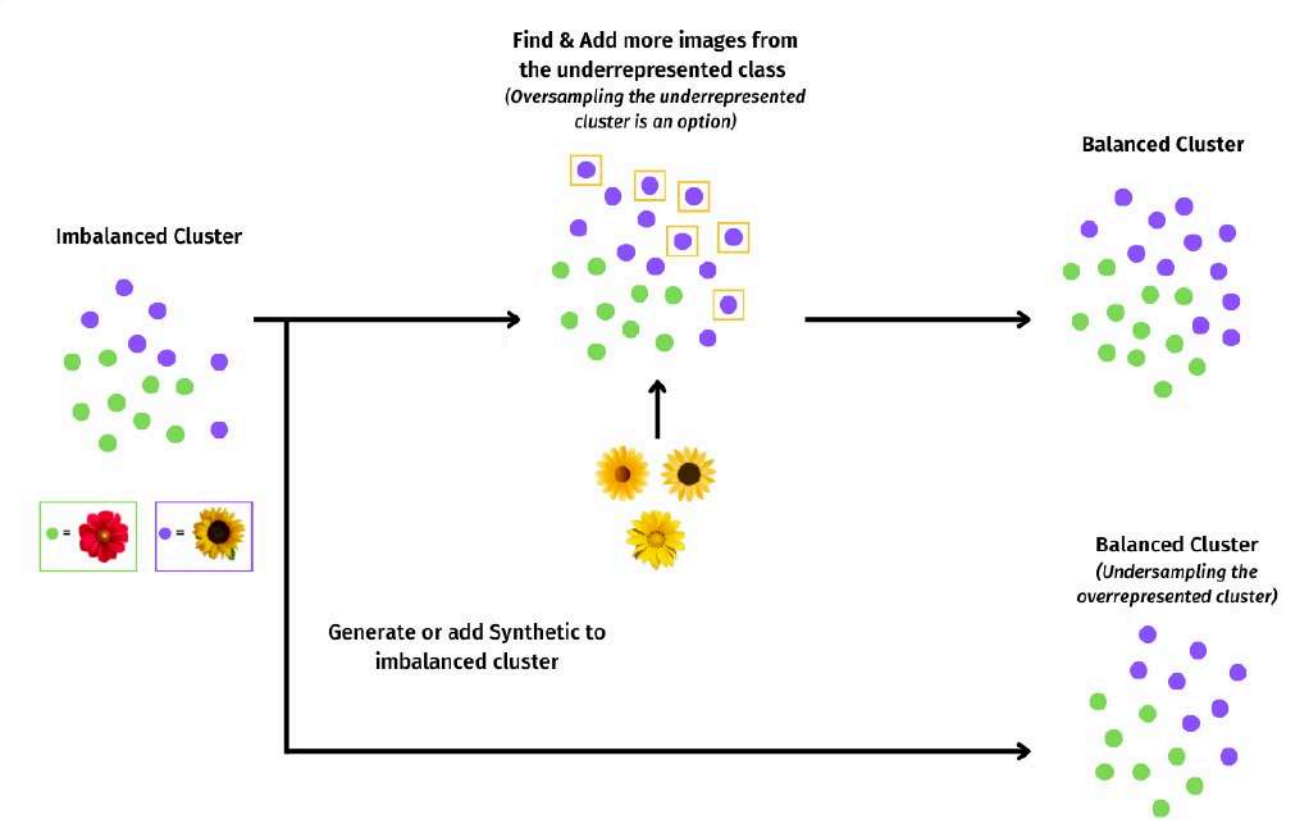
Why Accuracy Alone Isn't Enough

The Problem with Accuracy

Imagine a fraud detection model that **never flags any transaction as fraud**. It would be 99% accurate — because 99% of transactions are legitimate.

But it would **completely fail** at its actual job.

> This is why we need Precision, Recall, and F1 Score in addition to Accuracy.



Classification Metrics Explained

Accuracy

% of all predictions that are correct. Best when classes are balanced.

Formula: (TP + TN) / Total

Precision

Of all predicted positives, how many were actually positive? Minimizes false alarms.

Formula: TP / (TP + FP)

Recall

Of all actual positives, how many did we catch? Minimizes missed cases.

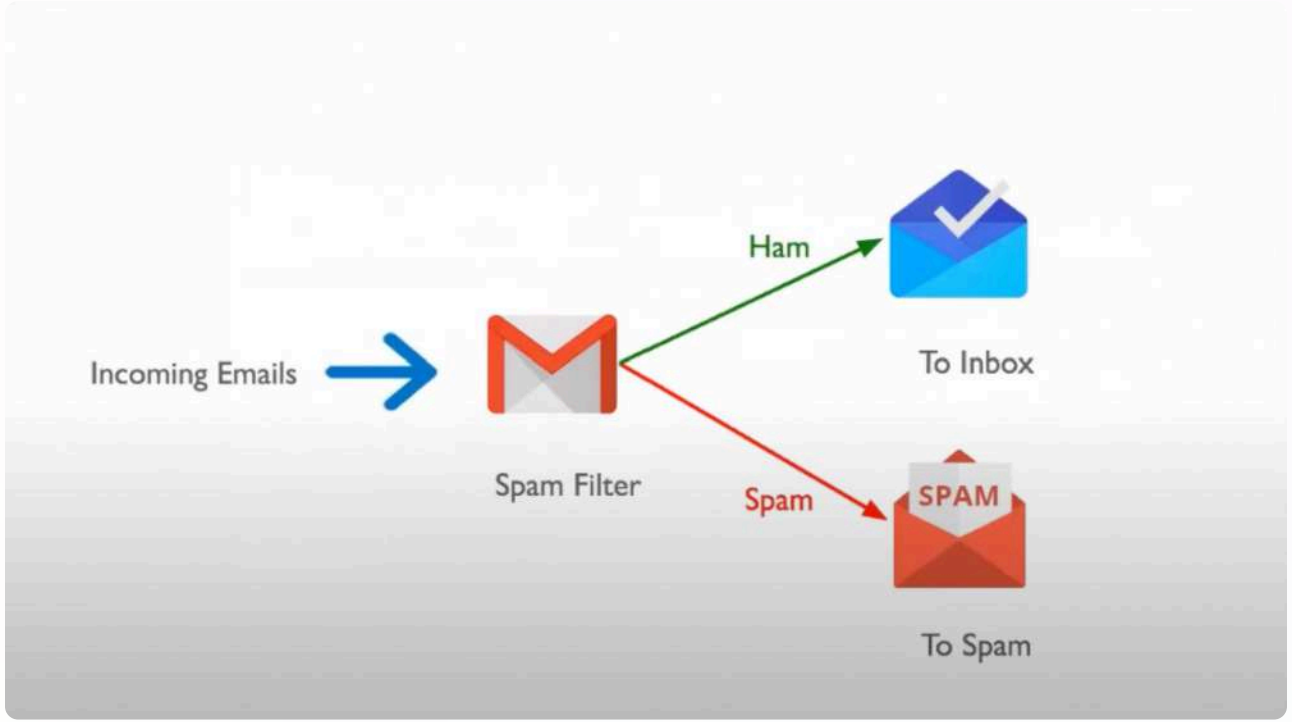
Formula: TP / (TP + FN)

F1 Score

Harmonic mean of Precision and Recall. Best single metric for imbalanced datasets.

Formula: 2 × (P × R) / (P + R)

Intuitive Example: Spam Detection



Applying the Metrics

- **High Precision**: When we say "spam," we're almost always right — few legitimate emails trashed
- **High Recall**: We catch most spam — very few slip into the inbox
- **F1 Score**: Balances both — ideal when missing spam AND false positives both matter

In medical diagnosis, Recall is critical — missing a cancer diagnosis costs lives.

Regression Metrics: RMSE & MAE

Mean Absolute Error (MAE)

Average of absolute differences between predicted and actual values. **Easy to interpret** — same units as the output.

Example: Model predicts house price off by $18,000 on average → MAE = $18,000

Root Mean Squared Error (RMSE)

Penalizes **large errors more heavily**. Best when big mistakes are especially costly.

Example: A few wildly wrong predictions inflate RMSE significantly

Choosing the Right Metric

1

Define the Problem

Classification or Regression?

2

Understand the Cost

False positives vs. false negatives — which is more costly?

3

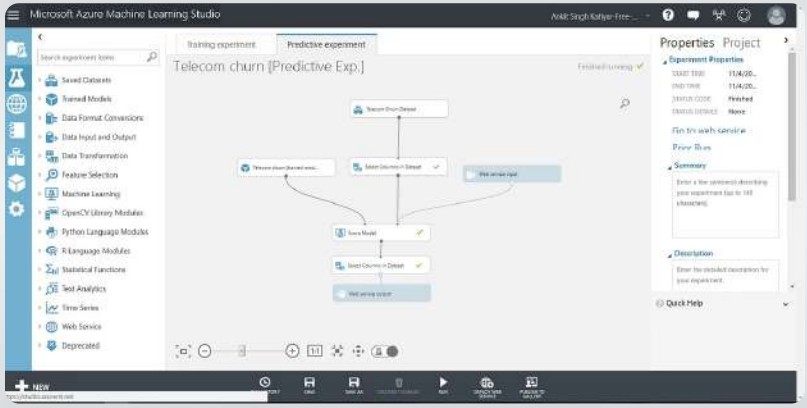
Pick the Metric

Accuracy, F1, RMSE, or MAE — aligned to your business goal

CHAPTER 4

Azure ML Experiment Tracking

Bringing reproducibility, comparability, and governance to your ML workflow



The Problem Experiment Tracking Solves

Without Tracking, Data Scientists Face:

- "Which model run gave me that 91% accuracy?"
- "What hyperparameters were used in production?"
- "I can't reproduce my best result from last week"
- No audit trail for model decisions in regulated industries

Azure ML Experiment Tracking solves all of this — automatically and at scale.

Core Concepts: Azure ML Experiment Tracking

Experiment

A named container grouping all related training runs for a specific ML task

Run

A single execution of a training script — captures all outputs automatically

Metrics Logging

Log accuracy, loss, AUC, and custom values during training using `run.log()`

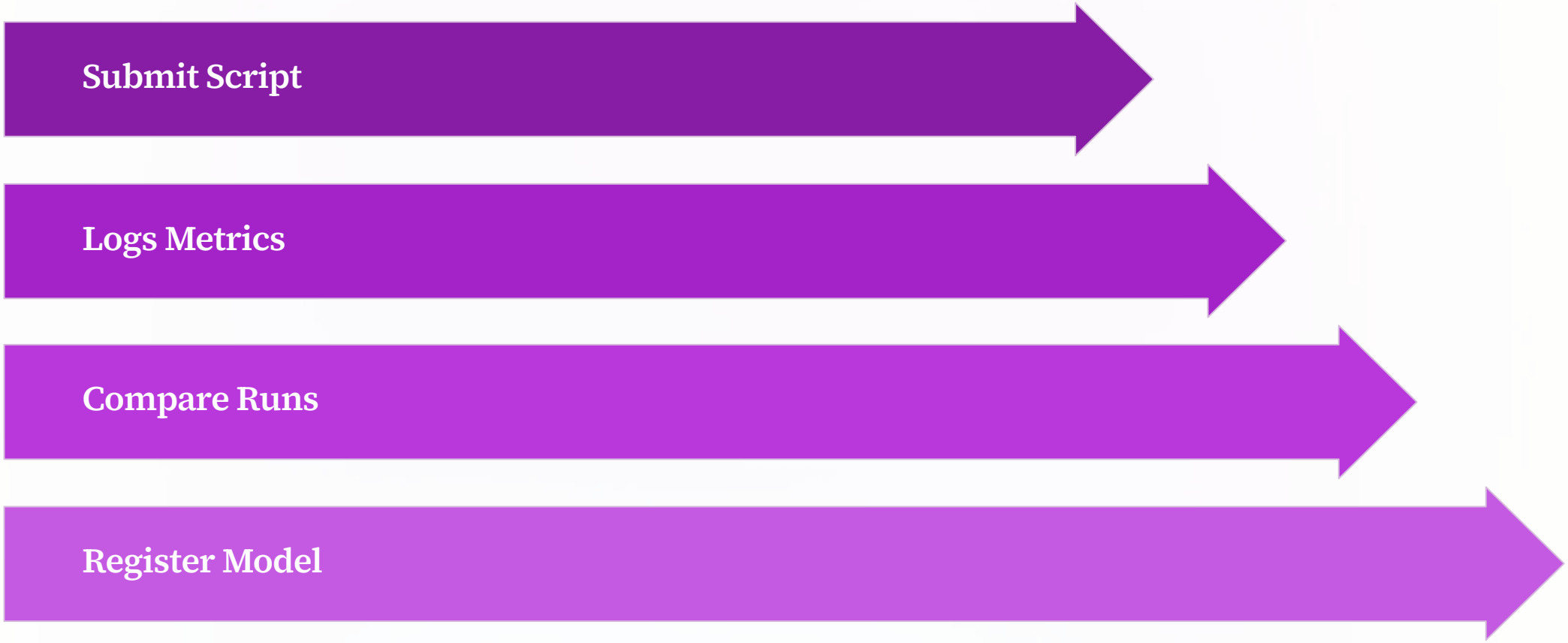
Model Logging

Save trained model artifacts to a run, making them retrievable and deployable

Model Versioning

Register models with version numbers — track what changed between v1, v2, v3

How Experiment Tracking Works



Every run automatically captures environment details, parameters, metrics, and output artifacts. The Azure ML Studio dashboard lets you compare runs side-by-side — no manual spreadsheet tracking needed.

Metrics Logging: Code in Practice

```
from azureml.core import Run

run = Run.get_context()

# Log a single metric
run.log("accuracy", 0.91)
run.log("f1_score", 0.88)

# Log a list of values over time
run.log_list("loss", [0.8, 0.6, 0.4])

# Upload the trained model
run.upload_file("outputs/model.pkl",
                "model.pkl")

run.complete()
```

What Gets Captured

- All `run.log()` values appear in Studio dashboard
- Training environment (Python version, packages)
- Start/end timestamps and duration
- Output files and model artifacts

Model Versioning in Azure ML

Model v1

Logistic Regression baseline — Accuracy: 82%

Model v2

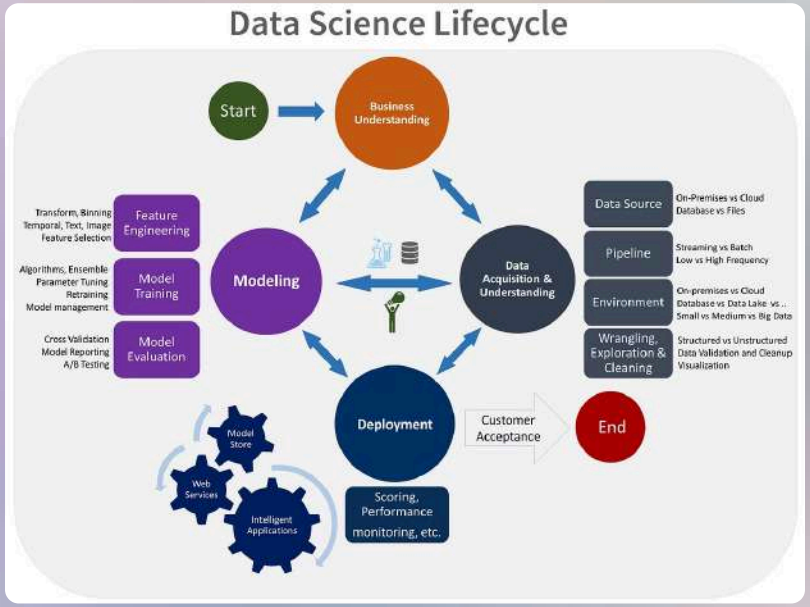
Random Forest — Accuracy: 89%, F1: 0.87

Model v3

Gradient Boosting + Tuning — Accuracy: 93%, F1: 0.92 ✅ Promoted to Production

Why Model Versioning Matters

- Roll back to a previous version if production degrades
- Audit trail for compliance and governance
- Compare versions objectively with logged metrics
- Tag versions with stage: **Staging → Production → Archived**



Value for Production ML Teams

Reproducibility

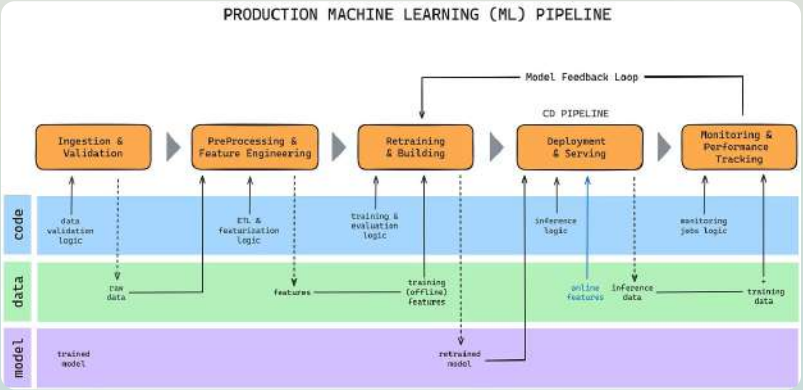
Recreate any run exactly — same data, code, environment, and parameters

Comparability

Compare dozens of runs on a single dashboard — pick the best model objectively

ML Lifecycle Governance

Full audit history from raw data to deployed model — critical for banking and healthcare

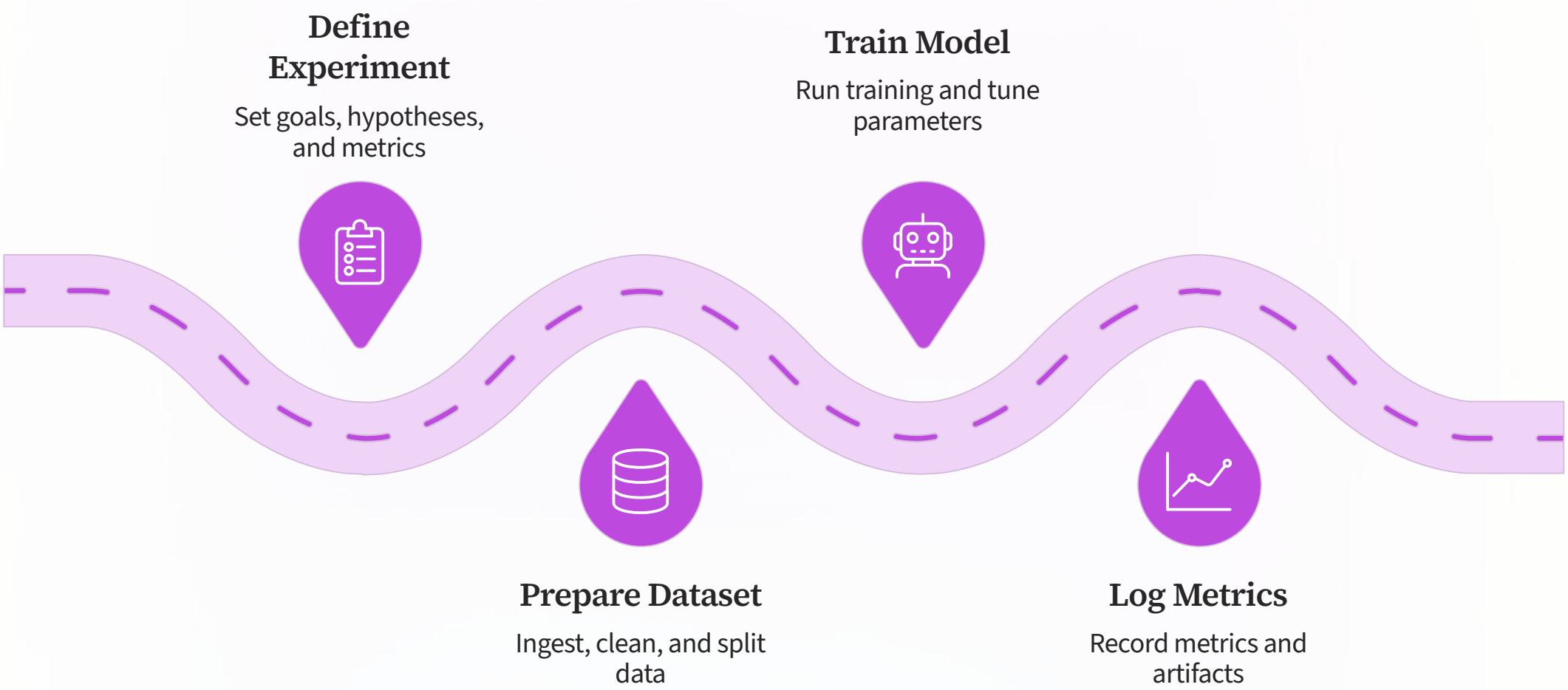


CHAPTER 5

The ML Experiment Lifecycle

From defining a business problem to deploying a registered model in production

The ML Experiment Lifecycle — Full Flow



This lifecycle applies across every Azure ML project — from a quick proof-of-concept to a production-grade enterprise pipeline. Each step maps directly to an Azure ML Studio feature you'll use in today's labs.

Lifecycle in Practice: Real Industry Examples

🏦 Banking — Fraud Detection

Define: Classify transactions as fraud or legit

Train: Gradient Boosting on 2 years of transaction history

Register: Deploy model v3 to real-time scoring endpoint

📱 Telecom — Churn Prediction

Define: Predict customers likely to cancel in 30 days

Train: Random Forest on usage patterns and support tickets

Register: Batch scoring pipeline runs nightly

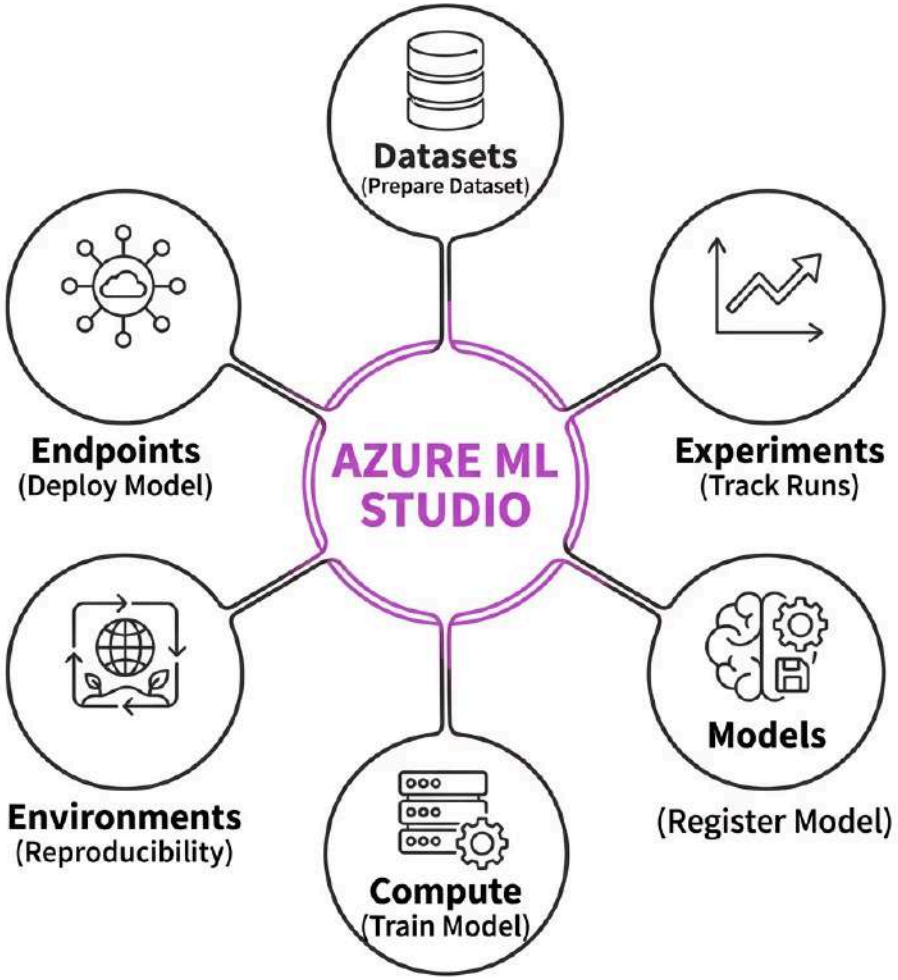
🛒 E-Commerce — Demand Forecasting

Define: Predict product sales for the next 2 weeks

Train: Regression model on seasonal sales data

Register: Model feeds inventory management system

Connecting the Lifecycle to Azure ML Tools



Everything Connected in One Platform

Azure ML Studio provides a unified workspace where every step of the lifecycle is traceable, auditable, and collaborative.

- No context-switching between tools
- Every artifact linked to the run that produced it
- Team members share the same workspace

Hands-On Labs

Time to apply everything — three labs covering training, tracking, and tuning

LAB SESSION — 3 HOURS

Lab Overview: Day 3 Hands-On Session

1

Lab 7

Train ML Model using Azure ML SDK

~60 min

2

Lab 8

Experiment Tracking & Run Comparison

~60 min

3

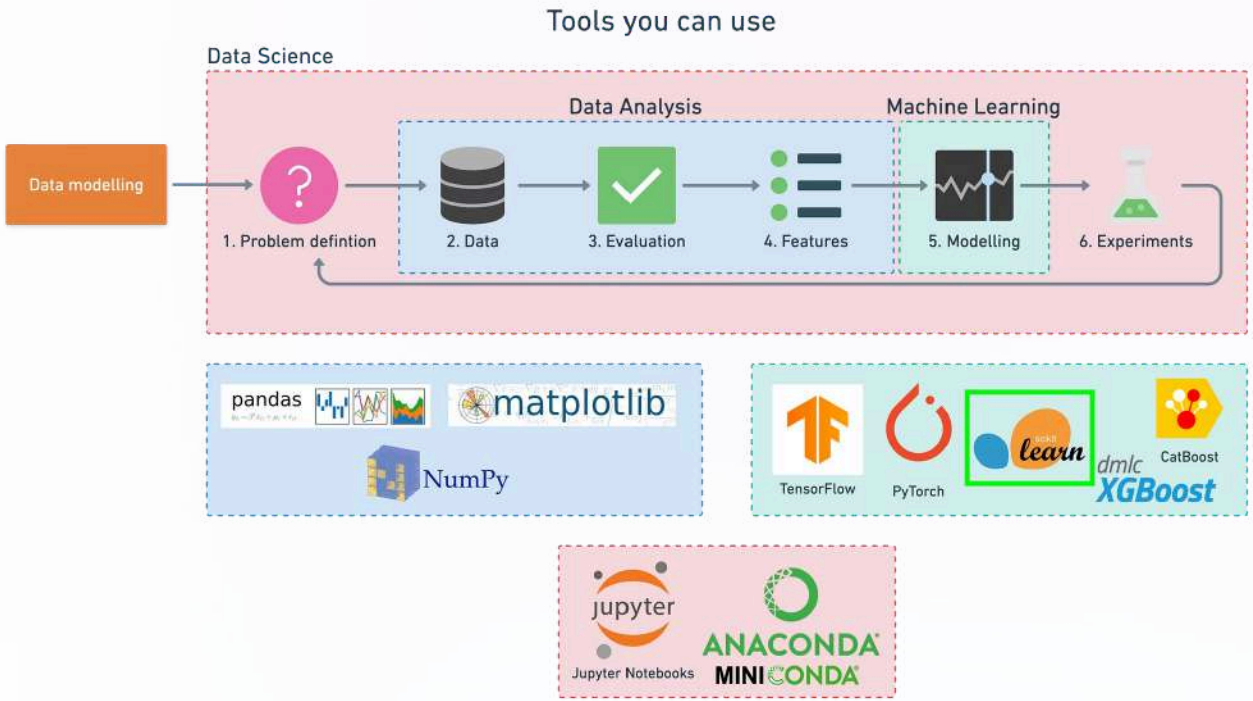
Lab 9

Hyperparameter Tuning with HyperDrive

~60 min

LAB 7

Train an ML Model Using Azure ML SDK



Objective

Train a **customer churn prediction model** using Python and scikit-learn, submitted as an Azure ML experiment run.

Business Context

A telecom company wants to predict which customers are likely to cancel their subscription in the next 30 days.

Lab 7 — Step-by-Step

01

Set Up the Training Script

Create `train.py` — load the churn dataset, preprocess features, and define a Random Forest classifier

02

Configure the Experiment Run

Use Azure ML SDK: define workspace, experiment name, compute target, and environment

03

Submit the Run

Submit the training script to Azure ML. Monitor real-time logs in Studio while the job executes

04

Log and Review Metrics

Use `run.log()` to capture Accuracy and F1 Score. View results in the Experiment dashboard

Lab 7 — Training Script Template

```
import argparse
from azureml.core import Run
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score
import pandas as pd

run = Run.get_context()

# Load data
df = pd.read_csv("churn_data.csv")
X = df.drop("churn", axis=1)
y = df["churn"]

# Train model
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)

# Log metrics
acc = accuracy_score(y_test, model.predict(X_test))
f1 = f1_score(y_test, model.predict(X_test))

run.log("accuracy", acc)
run.log("f1_score", f1)

run.complete()
```

Lab 7 — What You'll See in Azure ML Studio

After Submission

- Run appears under **Experiments** in the left nav
- Metrics panel shows logged `accuracy` and `f1_score`
- Outputs tab contains the saved model file
- Logs tab shows full training output

> Tip: Always verify the run completes successfully before moving to Lab 8.

LAB 8

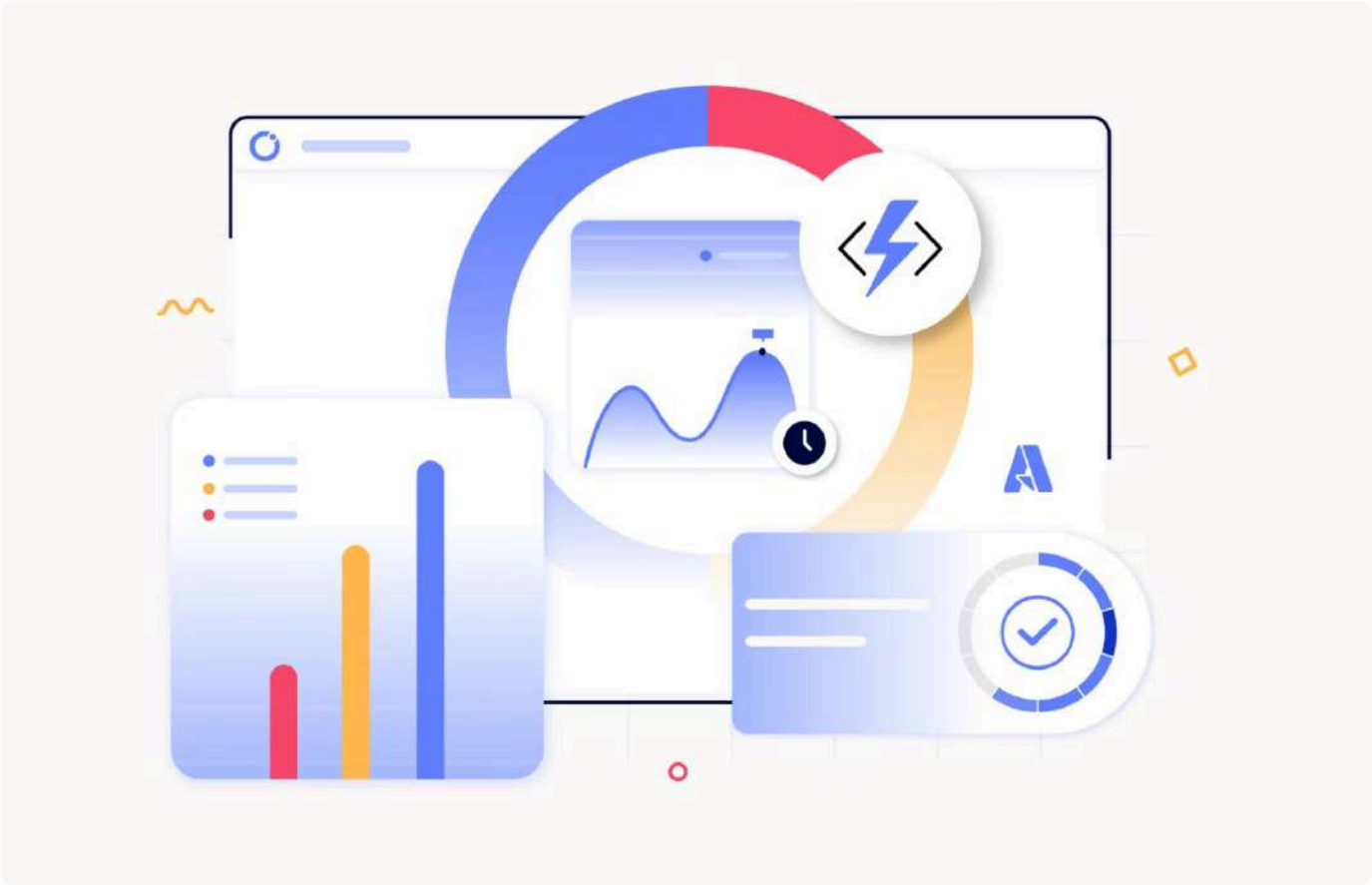
Experiment Tracking & Run Comparison

Objective

Run the churn model experiment **multiple times** with different configurations. Use the Azure ML Studio dashboard to compare runs and identify the best performing model.

Key Skills

- Multi-run experiment management
- Side-by-side run comparison
- Understanding ML reproducibility



Lab 8 — Three Runs, Three Configurations

Run 1: Baseline

Random Forest

n_estimators = 50

No preprocessing

Run 2: More Trees

Random Forest

n_estimators = 200

No preprocessing

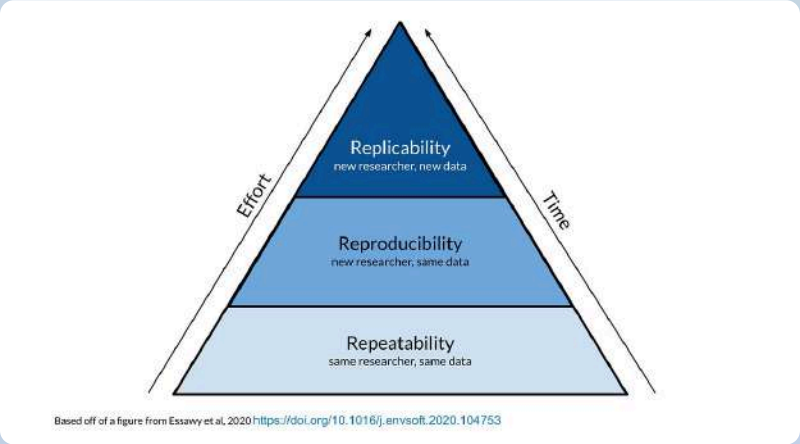
Run 3: Full Pipeline

Random Forest

n_estimators = 200

Feature scaling + encoding

After submitting all three runs, use the **Compare Runs** view in Azure ML Studio to chart metrics across all runs simultaneously — and select the winner.



ML Reproducibility — Why It Matters

Set a Random Seed

Use `random_state=42` in scikit-learn to ensure the same split and model initialization every run

Version Your Data

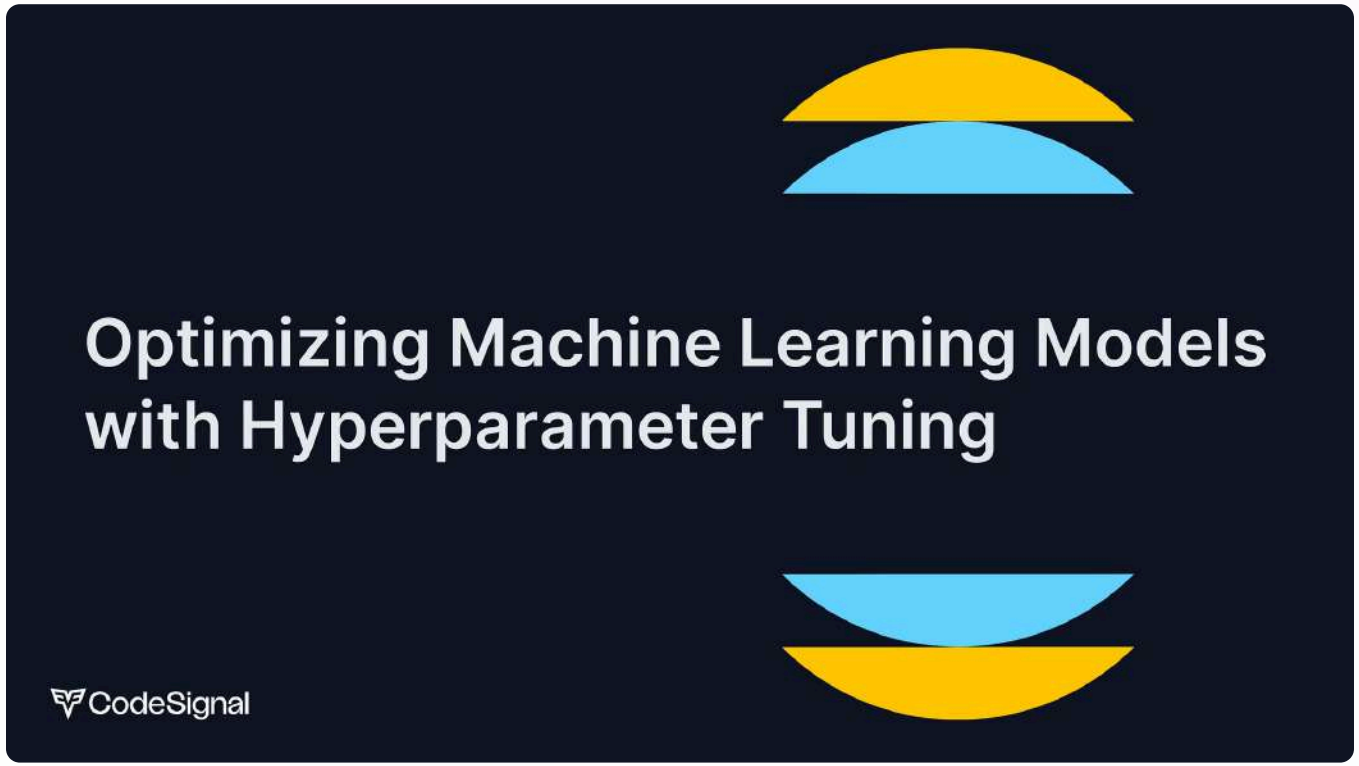
Register datasets in Azure ML so the exact data version used in each run is recorded

Pin Your Environment

Use Azure ML Environments with pinned package versions — no surprise behavior from library updates

LAB 9

Hyperparameter Tuning with HyperDrive

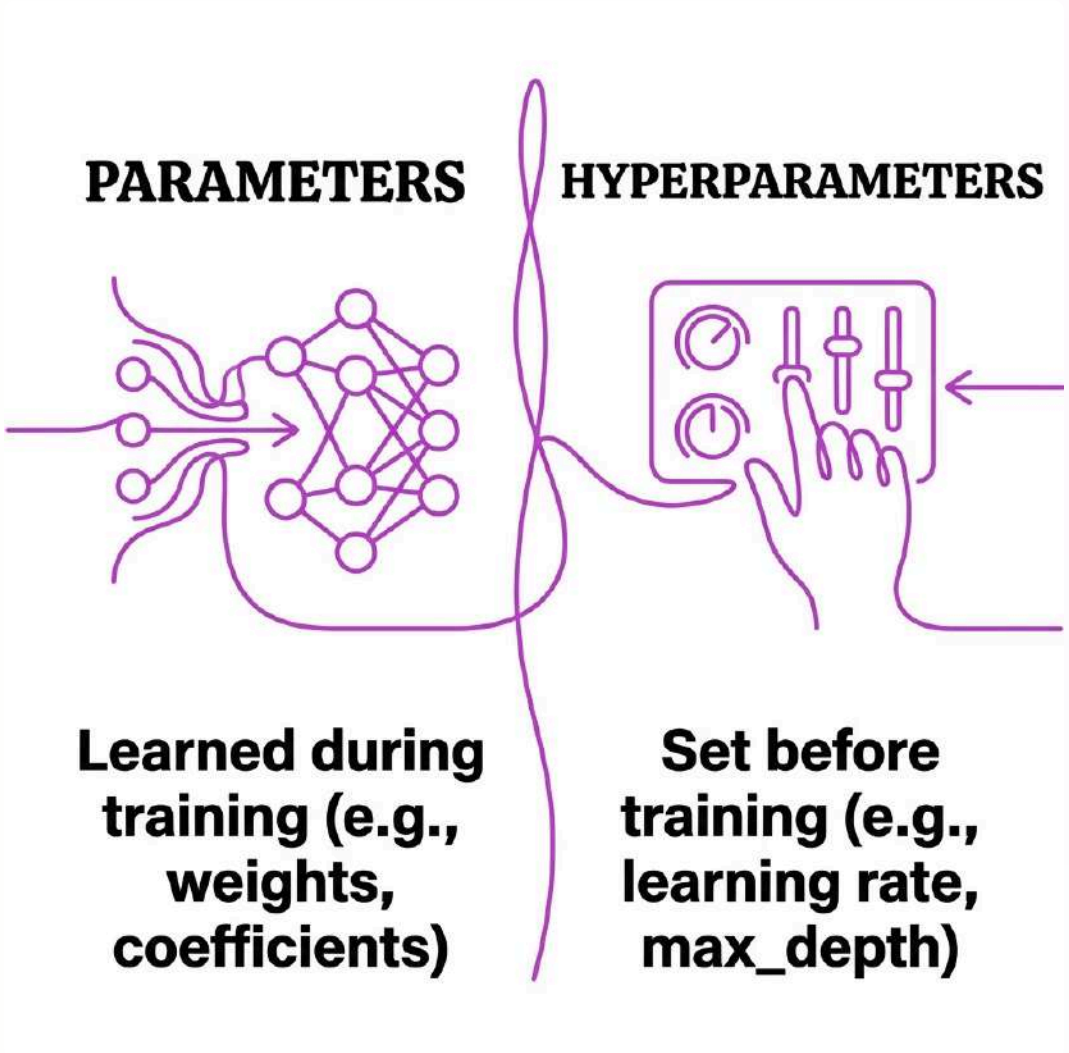


What Is HyperDrive?

HyperDrive is Azure ML's automated **hyperparameter tuning service**. Instead of manually testing combinations, it intelligently searches the parameter space to find the best model configuration.

HyperDrive can run dozens of trials in parallel — saving hours of manual experimentation.

What Are Hyperparameters?



Common Hyperparameters to Tune

- **n_estimators**: Number of trees in a forest (e.g., 50–500)
- **max_depth**: Maximum tree depth (e.g., 3–20)
- **learning_rate**: Step size for gradient boosting (e.g., 0.01–0.3)
- **C**: Regularization strength for Logistic Regression

HyperDrive Sampling Strategies

Grid Sampling

Tries every combination in a defined grid. Thorough but expensive for large spaces.

Random Sampling

Randomly samples the parameter space. Great balance of speed and coverage. Best for most use cases.

Bayesian Sampling

Uses past results to guide the next trial intelligently. Most efficient — fewer runs needed.

Lab 9 — HyperDrive Configuration

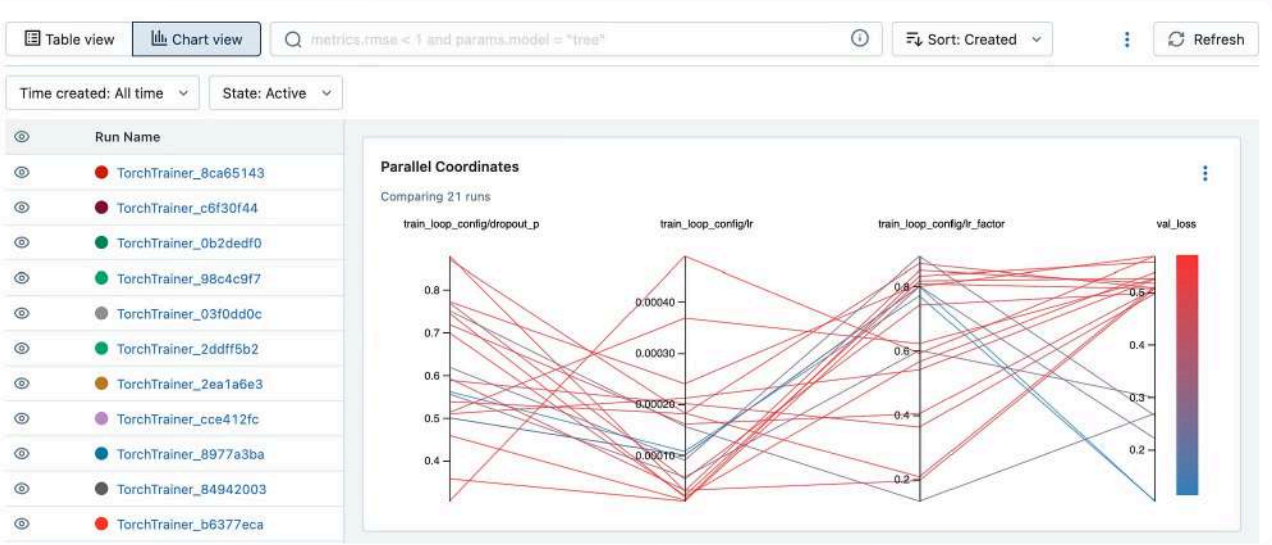
```
from azureml.train.hyperdrive import (
 HyperDriveConfig, RandomParameterSampling,
 BanditPolicy, PrimaryMetricGoal
)
from azureml.train.hyperdrive import uniform, choice

param_sampling = RandomParameterSampling({
 "n_estimators": choice(50, 100, 200),
 "max_depth": choice(3, 5, 10, None)
})

early_termination = BanditPolicy(
 slack_factor=0.1, evaluation_interval=1
)

hd_config = HyperDriveConfig(
 run_config=run_config,
 hyperparameter_sampling=param_sampling,
 policy=early_termination,
 primary_metric_name="accuracy",
 primary_metric_goal=PrimaryMetricGoal.MAXIMIZE,
 max_total_runs=20,
 max_concurrent_runs=4
)
```

Lab 9 — Reading HyperDrive Results



After HyperDrive Completes

- Studio shows a **parallel coordinates chart** — visualize how each parameter affects accuracy
- Best run is automatically highlighted
- Register the best model directly from the HyperDrive run

Use `best_run = hd_run.get_best_run_by_primary_metric()` to retrieve the winning configuration programmatically.

CHAPTER 6

Cost Optimization Tips for Training Labs

Smart resource habits during labs protect your Azure subscription budget — and build good production hygiene.

5 Ways to Reduce Lab Costs

1. **Use CPU Compute, Not GPU**

   For tabular ML with scikit-learn, CPU clusters (Standard_DS2_v2) are sufficient and 5–10× cheaper than GPU instances

2. **Limit HyperDrive Max Runs**

   Set `max_total_runs=20` during labs. Use Bandit early termination to kill poor-performing runs automatically

3. **Stop Compute Instances After Each Lab**

   A running compute instance billed when idle is the #1 source of unexpected charges. Stop it immediately after each lab

4. **Set Minimum Nodes to 0**

   Configure compute clusters with `min_nodes=0` so they scale down to zero when not actively running jobs

5. **Monitor Spend with Azure Cost Management**

   Set a budget alert at $50 during the training week — get notified before costs spiral

Cost Optimization Quick Reference

✅ DO

- CPU clusters for tabular ML
- min_nodes=0 on clusters
- Stop instances post-lab
- Use Bandit early termination
- Set budget alerts

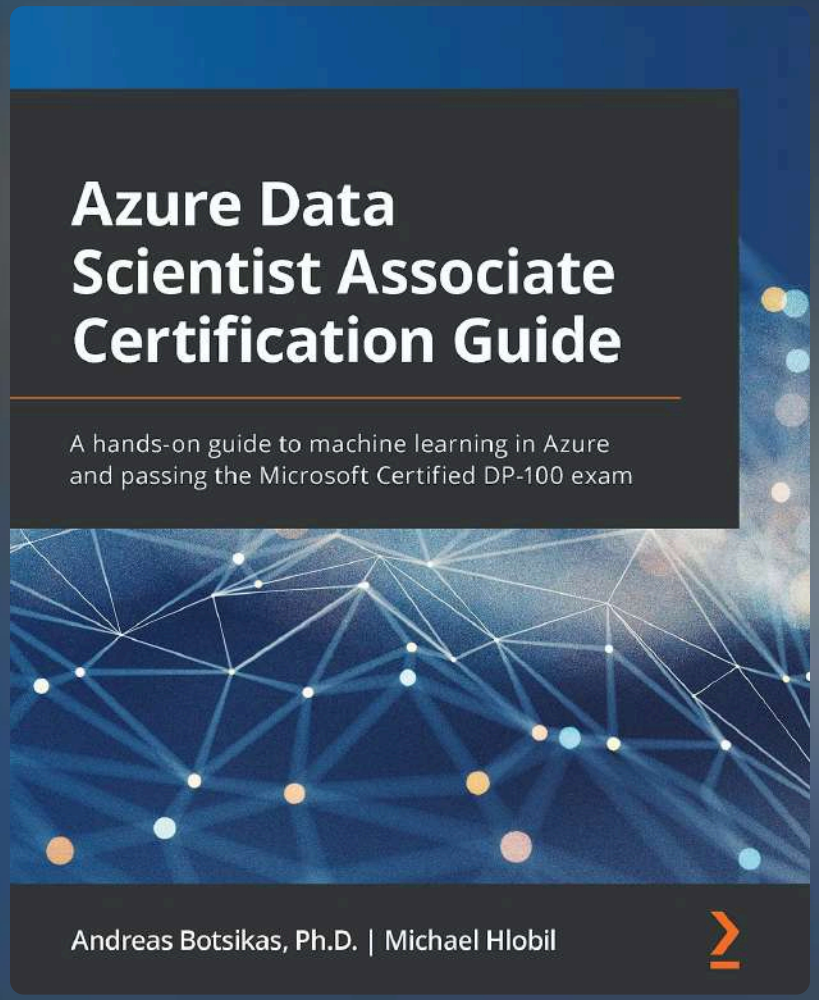
❌ DON'T

- Leave compute running overnight
- Use GPU for scikit-learn workloads
- Run unlimited HyperDrive trials
- Skip monitoring the cost dashboard

DAY 3 WRAP-UP

Key Takeaways

What you learned and built today — and how it connects to DP-100



Day 3 Key Takeaways

ML Algorithms

Logistic Regression, Decision Trees, Random Forest, and Gradient Boosting each serve specific problem types — algorithm selection is a business decision, not just a technical one

Evaluation Metrics

Accuracy, Precision, Recall, F1 for classification; RMSE and MAE for regression. Always choose metrics aligned to the real cost of errors in your domain

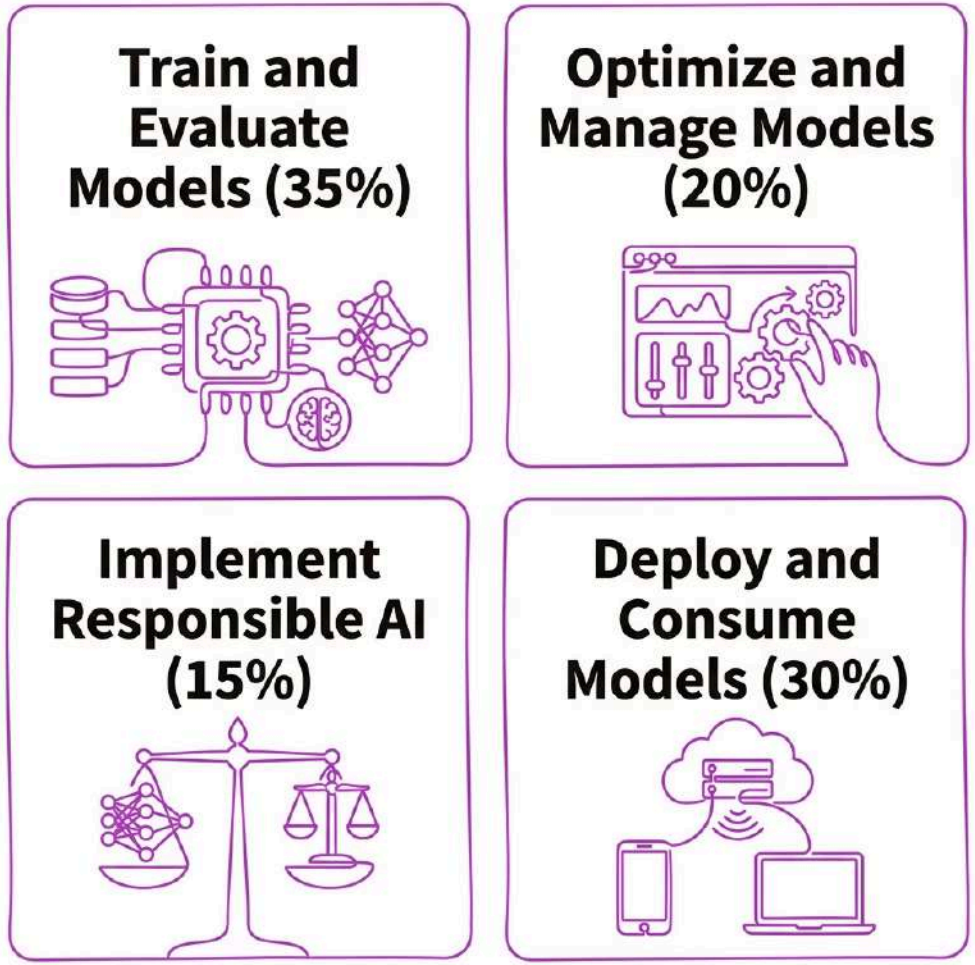
Experiment Tracking

Azure ML automatically logs every run. Use `run.log()` for custom metrics, compare runs in Studio, and register the best model with version history

HyperDrive Tuning

Automates the search for optimal hyperparameters using Random, Grid, or Bayesian sampling — with early termination to save compute costs

How Day 3 Maps to DP-100 Exam Objectives



DP-100 Skills Reinforced Today

- Use Azure ML SDK to run training experiments
- Select and apply appropriate ML algorithms
- Evaluate model performance using appropriate metrics
- Track, compare, and register models in Azure ML
- Tune hyperparameters using HyperDrive

Preview: Day 4

Model Deployment

Deploy trained models as real-time endpoints using AKS and ACI

ML Pipelines

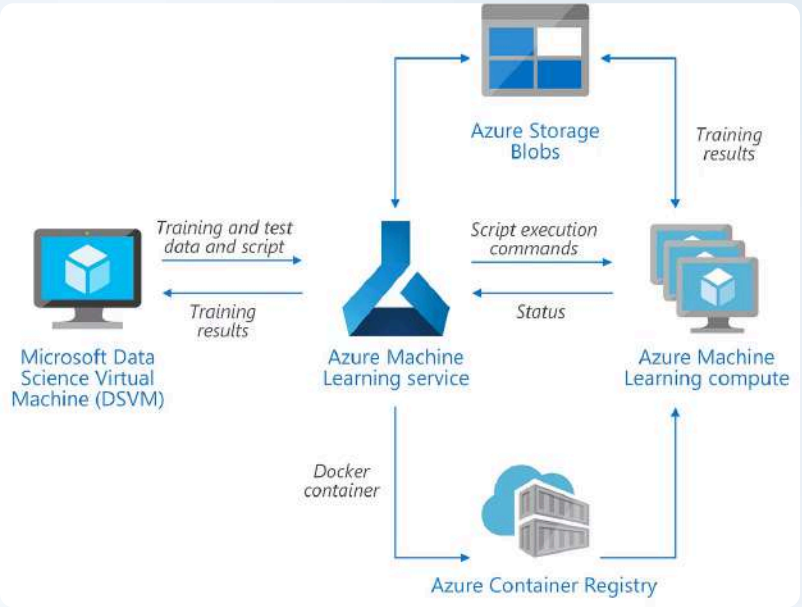
Build automated, reusable training pipelines with Azure ML Pipeline SDK

Model Monitoring

Detect data drift and performance degradation in production models

Before Day 4: Ensure all three labs are complete and your compute instances are **stopped**.

Great Work Today!

You've trained your first Azure ML models, tracked experiments like a production data scientist, and automatically tuned hyperparameters with HyperDrive.

Remember to stop your compute instances before you leave. See you for Day 4!